

Sleep-time scaling law는 어떻게 세워야 하는가

어려운 Study paper 를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study 의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper 의 해당 개념으로 연결된다. 주장 ID 와 figure ID 는 원 Study 와 동일하다.

Contents

E09 · Sleep-time scaling law 는 어떻게 세워야하는가	2
이 권이 답할질문	2
먼저 세 문장	2
직관	2
예와 반례	3
그림 읽기	4
대안과 비교	4
아직 모르는것	5
더 깊이 읽기	5
문서 경계와 사용법	8

E09 · Sleep-time scaling law 는 어떻게 세워야하는가

이 권이 답할질문

Pretraining 에는 compute·data·parameter 와 loss 의 경험적 scaling law 가있다. Sleep-time compute 에도 “더 오래 자면 더 똑똑해진다” 는 법칙이생길까? 현재 답은 **아직 universal law 가 없다**. 먼저 cost accounting identity, reuse break-even, cadence optimum, capacity knee 를 분리하고, 이후 workload 별 empirical fit 을 쌓아야 한다.

이 권은 scaling-law section 을 풀며 **STC-C041** · synthesis · confidence high, **STC-C043** · hypothesis · confidence medium, **STC-C044** · hypothesis · confidence medium, **STC-C045** · hypothesis · confidence medium 을 구분한다.

먼저 세 문장

1. Raw sleep FLOP, 처리한 memory 수, 생성한 dream token 은 좋은 scaling variable 이 아니다. 잘못된 기억을 많이 처리하면 utility 가 음수가 될 수있다.
2. 핵심 후보는 **valid reusable evidence per unit sleep cost** 다. Validity, reuse, retention, staleness, interference 를 함께 측정해야한다.
3. Sleep cadence 와 memory capacity 에는 interior optimum 이 있을 가능성이높다. 너무 자주 갱신하면 overhead·noise 가 크고, 너무 늦으면 staleness 가 커진다.

직관

Identity 와 law 를구분한다

다음은 정의에 가까운 accounting identity 다.

$$C_{\text{sleep}} = C_{\text{read}} + C_{\text{transform}} + C_{\text{train}} + C_{\text{eval}} + C_{\text{publish}}$$

각 항을 더하면 총비용이 된다는 것은 항상 참이지만 성능이 어떻게변하는지는 말하지 않는다. Scaling law 라면 여러 규모에서 측정한 data 로예측력이 있어야 한다. 예를 들어 utility gap 이 sleep compute 의 power law 로 줄어드는지, 어느 점에서 saturation 하는지 fit 하고 out-of-sample 에서맞아야 한다.

Break-even 은 reuse 문제다

External evidence 를 매번 retrieve 하는 비용을 c_r , 한 번 sleep/compile 하는 비용을 c_s , compile 후 매 query 절약을 Δc 라 하자. 단순 break-even reuse 는 다음처럼 생각할 수 있다.

$$N^* \approx \frac{c_s + c_{\text{governance}}}{\Delta c}$$

$N > N^*$ 일 때만 compile 이 경제적이다. 하지만 quality loss 와 rollback risk 를 cost 에 포함해야 한다. 이 식은 measurement model 이지검증된 universal law 가 아니다.

좋은 분모와 좋은분자

분모에는 GPU FLOP 만 넣지 않는다. Joule, wall-clock, accelerator-hour, HBM/SSD bytes moved, index write amplification, validation inference 를기록한다. 분자에는 raw memory count 대신 future task 에서 실제로 사용된 **valid retained evidence** 를 둔다.

한 후보 metric 은 다음과 같다.

$$\eta_{\text{STC}} = \frac{U_{\text{future}} \times R_{\text{retained}} \times P_{\text{valid}}}{C_{\text{sleep}} + C_{\text{serve overhead}} + C_{\text{rollback}}}$$

여기서 U 는 future utility, R 은 retention, P 는 provenance/validation 을 통과한 비율이다. 이 역시 law 가 아니라 연구를위한 normalization proposal 이다.

Cadence optimum

Sleep interval τ 가 작으면 fresh 하지만 job launch 와 validation 이너무 자주 일어난다. τ 가 크면 batch efficiency 는 좋아지지만 memory 가 stale 해지고 wake state 가 넘칠 수 있다. Traffic 과 update entropy 가 변하므로 optimal τ 는 constant 가 아닐 가능성이 높다.

Capacity knee

Memory byte M 을 늘릴 때 초반 utility 는 증가하지만 중복 ·interference·retrieval noise·compaction cost 때문에 marginal gain 이떨릴 수 있다. 용량예산의 knee 를 넘으면 더 큰 device 보다 admission·eviction policy 가중요해진다.

예와 반례

예 — Adapter compile break-even 실험

동일한 knowledge bundle 을 RAG 로만 제공하는 system 과 adapter 로 compile 한 system 을 비교한다. Reuse 횟수를 1, 10, 100, 1000 으로 바꾸고 answer quality, TTFT, energy, adapter load, delete cost 를 쟀다. Break-even 은 quality constraint 를 만족하는 최초 reuse 지점이다. Hardware 가 바뀌면 N^* 도 바뀐다.

예 — Cadence sweep

Sleep interval 을 1 분, 1 시간, 1 일, 1 주로 바꾸고 staleness, batch size, queue delay, wake SLA interference, derived-memory error 를 측정한다. U-shaped total cost 가 나오면 interior optimum 가설이 지지된다. 단일 cadence 의 SOTA 만 보고 law 라고 부르면 안 된다.

반례 — Dream token 을 10 배 늘렸더니 benchmark +1

한 점의 결과는 scaling law 가 아니다. 다른 model size, data quality, recursive depth, task 에서 반복되고 held-out scale 을 예측해야 한다. 또한 extra token 이 중복이면 유효 evidence 는 거의 늘지 않았을 수 있다.

수식의 지위를 섞지 않는다

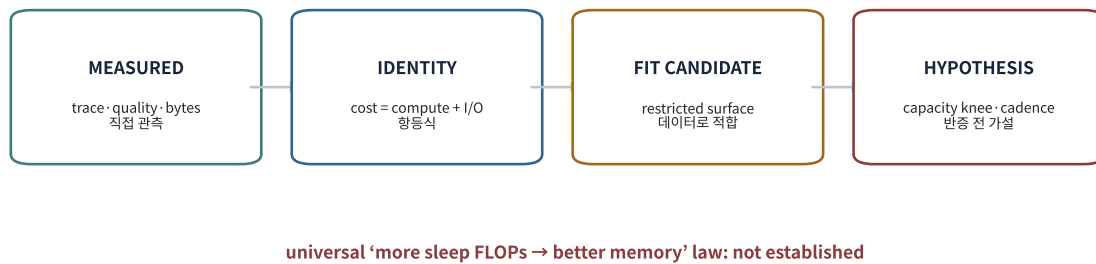


Figure 1: STC-F007. Measured identities, break-even models, and hypotheses. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 — Memory byte 가 크면 capacity 문제해결

Storage capacity 를 늘려도 active index, cache hit, retrieval precision, write endurance 가 병목일 수 있다. 유한한 latency 와 energy 안에서 usable capacity 를 재야 한다. “저장 가능한 byte” 와 “정확히 찾아 활용 가능한기억” 은 다르다.

그림 읽기

Figure 는 measured identity, break-even model, fit candidate, untested hypothesis 를 색으로 분리한다. 수식이 있다는 이유만으로 theory 가 되는것을 막기 위한 장치다. 연구 보고서에는 각 equation 의 status 를 함께적어야 한다.

Sleep cost 는 한 번 앞에서 지불하고 later reuse 에서 절약을 얻는다. Curve 가 0 을 지나는 지점이 break-even 이다. External memory maintenance 처럼 read cost 를 줄이지 않고 quality 를 높이는 경우에는 절약이 아니라 utility gain 을 monetary/energy-equivalent 로 환산해야 한다.

Capacity knee 는 finite-memory system 의 saturation 가설이다. Knee 이전에는 byte 추가가 유용하고, 이후에는 compaction·eviction·tiering 이 더중요하다. Device solution 은 knee 를 오른쪽으로 옮기거나 knee 이후 비용을 낮추는 제품으로 정의할 수 있다.

Cadence frontier 는 staleness 와 batching 의 tradeoff 를 보여 준다. Fixed daily sleep 보다 event-triggered/learned cadence 가 frontier 를 개선할 수있다. 그러나 scheduler 를 학습하려면 delayed future utility 를 현재 sleep decision 에 연결하는 평가가 필요하다.

대안과 비교

후보 관계	현재 지위	필요한 축	반증 조건
sleep FLOP ↑ → utility ↑	미검증	model, data quality, task	동일 compute 에서 saturation/하락
reuse ↑ → compile 가치 ↑	break-even model	access count, latency, quality	governance cost 가 절약 초과

후보 관계	현재 지위	필요한 축	반증 조건
cadence 에 interior optimum	가설	staleness, batch, SLA	boundary 가항상 최적
capacity knee 존재	가설	byte, entropy, interference	marginal utility 가 계속 일정
valid evidence efficiency	metric proposal	utility, validity, cost	measurement 가 불안정

오프폴리시평가는 실제로 다른 cadence 를 배포하지 않고 logged policy 에서 utility 를추정하는 후보지만, hidden confounding 과 distribution shift 에 취약하다. 초기에는 controlled sweep 과 replay simulation 이 필요하다.

아직 모르는것

- Future utility 를 memory write 에 귀속하는 credit assignment 가미해결이다.
- External synthesis 와 parametric training 의 cost unit 을 공정하게통일하기 어렵다.
- Quality·safety·delete 를 하나의 objective 로 결합하면 weight 선택에 따라결론이 달라진다.
- Model size 가 커질 때 sleep sample efficiency 가 좋아지는지 나빠지는지공개 scaling sweep 이 없다.
- Capacity knee 가 device bandwidth, index algorithm, memory entropy 중무엇에 더 민감한지 모른다.

핵심

첫 STC scaling law 는 “sleep FLOP law” 보다 “재사용되는 유효 evidence 와 lifecycle 총비용의 관계” 에서 나올 가능성이 높다.

더 깊이 읽기

- 원 분석: accounting identity·break-even·hypothesis 의 엄격한 구분
- Training Background: 목적함수, 검증세트, 용량예산, 울-왜곡, 오프폴리시평가
- Claim route: **STC-C041** · synthesis · confidence high · **STC-C043** · hypothesis · confidence medium · **STC-C044** · hypothesis · confidence medium · **STC-C045** · hypothesis · confidence medium

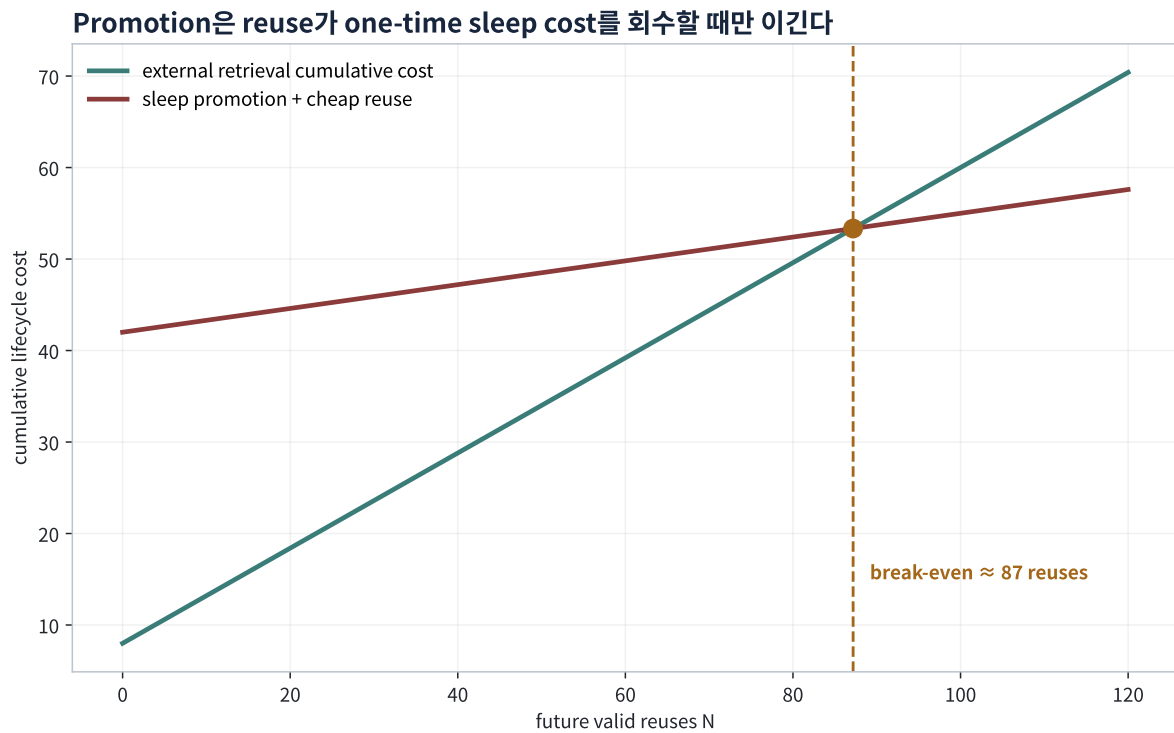


Figure 2: STC-F008. Sleep-cost versus later-reuse break-even. 원 Study 의저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

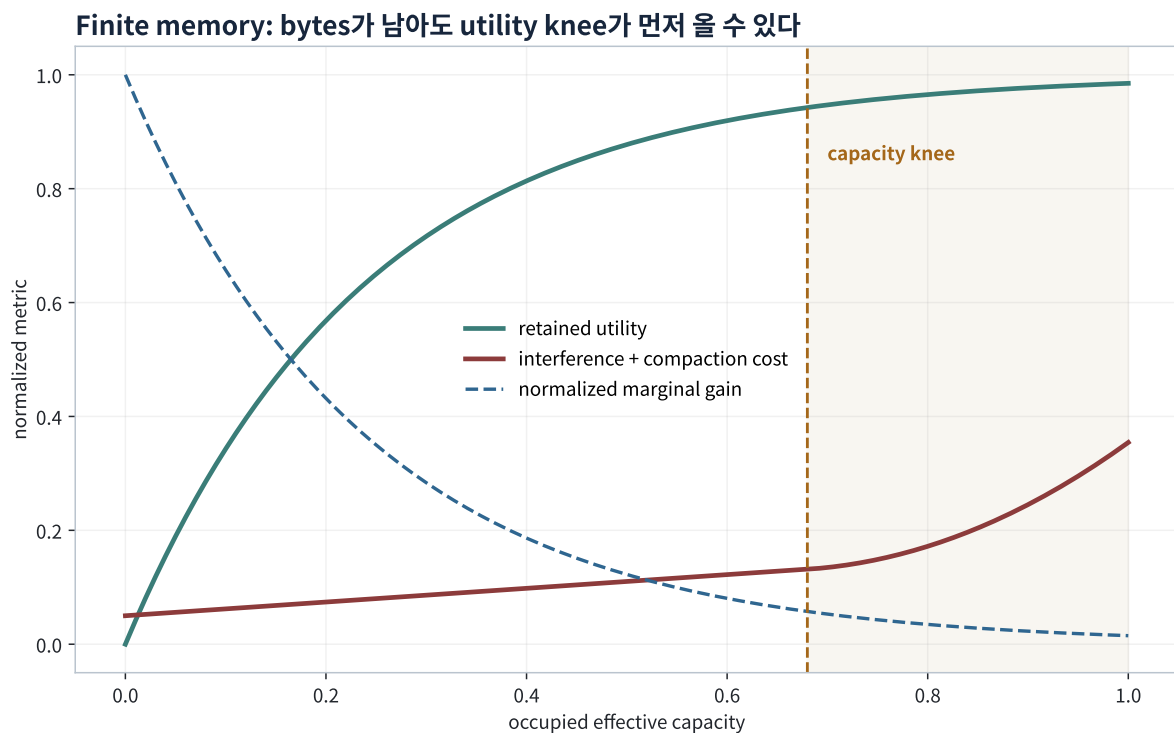


Figure 3: STC-F009. Finite-memory capacity knee. 원 Study 의 저자 작성합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

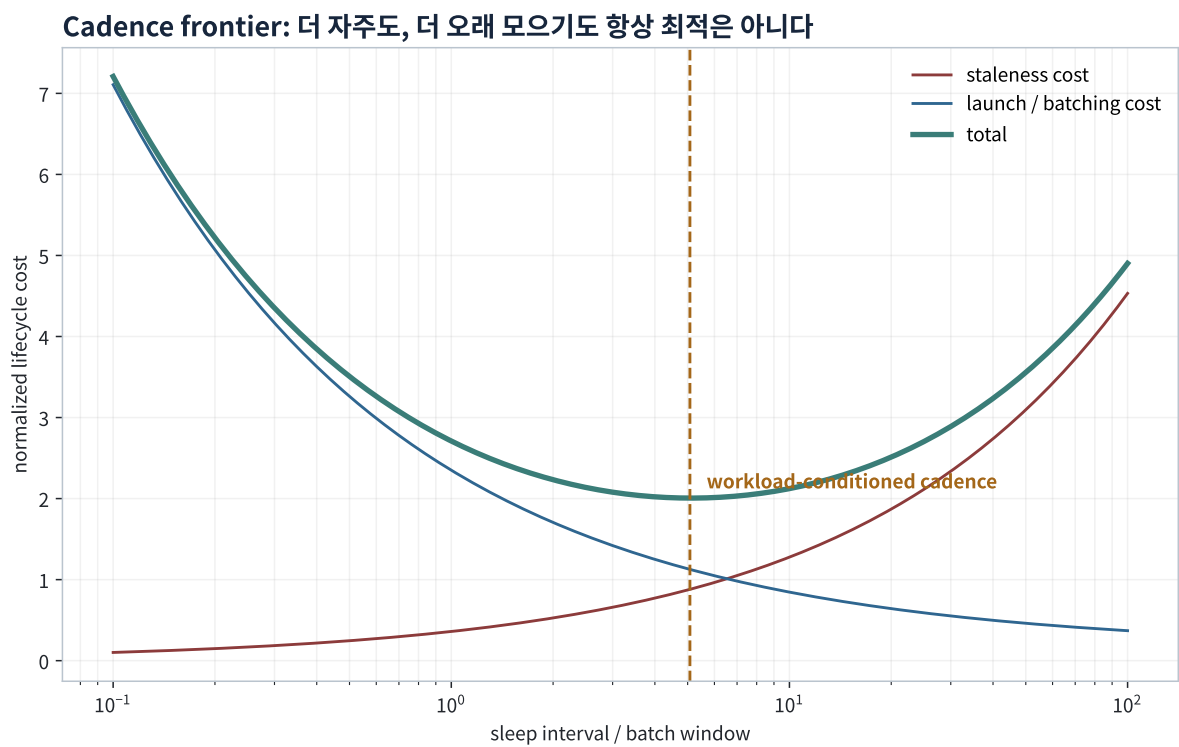


Figure 4: STC-F010. Cadence frontier across staleness and batching. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.